Journal of King Saud University Computer and Information Sciences (2025) 37:330
https://doi.org/10.1007/s44443-025-00353-3

ORIGINAL PAPER

# Adaptive heterogeneous multi-agent debate for enhanced educational and factual reasoning in large language models

**Yan Zhou**[1] · **Yanguang Chen**[2]

Received: 14 August 2025 / Accepted: 18 October 2025 / Published online: 24 November 2025
© The Author(s) 2025

**Abstract**
Large language models (LLMs) have achieved impressive results in complex reasoning and knowledge tasks, yet they often struggle with factual accuracy and logical consistency. Prior works have improved LLM performance using prompt-based techniques (e.g. chain-of-thought prompting and self-consistency) and post-hoc self-refinement, but these typically operate on a single model instance. Recently, multi-agent debate frameworks have emerged as a complementary approach, wherein multiple LLM agents propose answers and critique each other's reasoning to reach consensus. Such a "society of minds" approach has been shown to significantly improve mathematical reasoning and reduce factual hallucinations. However, existing debate methods use homogeneous agents with simple majority voting, limiting their effectiveness. In this work, we propose Adaptive Heterogeneous Multi-Agent Debate (A-HMAD), a novel framework that extends multi-agent debate with (i) diverse specialized agents, (ii) dynamic debate routing, and (iii) a learned consensus mechanism. Each agent in A-HMAD is assigned a distinct role or expertise (e.g. logical reasoning, factual verification, strategic planning), enabling more comprehensive error-checking and perspective diversity than identical agents. A coordination policy dynamically selects which agents contribute at each round based on the question's domain and the evolving debate state. To aggregate viewpoints, we introduce a consensus optimizer that learns to weight each agent's vote according to its reliability and the confidence of its arguments. On six challenging benchmarks – including arithmetic QA, grade-school math (GSM8K), multifact question answering (MMLU), factual biography generation, and chess strategy – our A-HMAD consistently outperforms prior single-model methods and the original multi-agent debate baseline. Notably, A-HMAD achieves 4–6% absolute accuracy gains over standard debate on these tasks, and reduces factual errors by over 30% in biography facts. We provide extensive ablations demonstrating the benefits of agent heterogeneity, additional debate rounds, and the learned consensus module. Our findings suggest that an adaptive, role-diverse debating ensemble can drive significant advances in LLM-based educational reasoning, paving the way for safer, more interpretable, and pedagogically reliable AI systems.

**Keywords** Heterogeneous multi-agent systems · Large language models · Factuality enhancement · Causal contrastive learning · Adaptive attention routing

## 1 Introduction

Large-scale pre-trained language models have revolutionized NLP, demonstrating remarkable prowess in tasks requiring knowledge and reasoning (Brown et al. 2020; Srivastava et al. 2023). Models like GPT-3 and its successors exhibit strong few-shot learning abilities (Brown et al. 2020) and can generate coherent multi-step reasoning chains when prompted appropriately (Kojima et al. 2022). Despite these gains, even state-of-the-art LLMs often produce hallucinations – confidently stated false facts – and illogical reasoning leaps, undermining their reliability (Lin et al. 2021; Ji et al. 2023). This is due in part to the noisy, unverified nature of web-scale training data and the lack of explicit reasoning verification in standard next-token prediction training (Ouyang et al. 2022; Ziegler et al. 2019). Improving the factual accuracy and reasoning soundness of LLM outputs has therefore become a pressing research goal (Lee et al. 2022; Kadavath et al. 2022).

✉ Yanguang Chen
2017212301@live.sufe.edu.cn

1 College of Mathematics and Informatics, South China Agricultural University, 510642 Guangdong, China

2 School of Information Management and Engineering, Shanghai University of Finance and Economics, 200433 Shanghai, China

A rich body of recent work has explored techniques to mitigate LLM reasoning and factual errors. Prompting strategies such as providing step-by-step exemplars or eliciting a chain-of-thought (CoT) have led to notable improvements in arithmetic and commonsense reasoning (Kojima et al. 2022; Wei et al. 2022). For example, Wei et al. (2022) showed that CoT prompting enables models to break down problems into logical steps, and Kojima et al. (2022) found that even zero-shot prompts (e.g. "Let's think step by step") can induce multi-step reasoning in LLMs. Building on CoT, self-consistency decoding samples multiple reasoning paths and selects the most common answer, which boosts correctness by averaging out reasoning variance (Wang et al. 2022). Other works equip LLMs with scratchpads or external tools: Nye et al. (2021) introduced "scratchpad" memory for intermediate computations, and tool-use approaches allow models to consult calculators or knowledge bases to check facts (Yao et al. 2023; Schick et al. 2023). Orthogonal to prompting, learning-from-feedback methods like reinforcement learning from human feedback (RLHF) fine-tune models to favor truthful, helpful outputs (Christiano et al. 2017; Stiennon et al. 2020), significantly reducing toxic or untrue generations (Siddiky et al. 2025). Likewise, post-hoc verifier models have been trained to check and correct a model's answers (e.g. for math word problems Cobbe et al., 2021). Another promising direction is self-reflection and refinement, where the model iteratively critiques and improves its own output (Kawamura et al. 2003; Madaan et al. 2023). Madaan et al. (2023) showed that prompting an LLM to reflect on potential errors and then attempt a revised solution can yield more accurate results, echoing human problem-solving strategies (Renze and Guven 2024). Despite these advances, most techniques above involve a single model reasoning in isolation; the model may still overlook certain errors or fail to verify all facts due to its inherent biases or blind spots (Kadavath et al. 2022; Wei et al. 2022).

In contrast to single-model approaches, multi-agent frameworks draw inspiration from social or argumentative processes in which multiple experts collaborate or debate to reach a better answer (Michael et al. 2023). Early conceptual work by Minsky (Michael et al. 2023) posited that intelligent behavior could emerge from a "Society of Mind" – a diverse collection of simpler agents. In the context of AI alignment and truthfulness, Khan et al. (2024) proposed AI debate, where two agents argue for opposing answers and a judge (human or AI) decides which argument is more convincing. The hope is that through adversarial debate, the truth would prevail because any false argument can be revealed by the opponent (Khan et al. 2024). While Irving's work was theoretical, recent empirical studies have started to realize the benefits of multi-agent interactions. For example, Li et al. (2023) introduced a communicative multi-agent system (Camel) where agents with different roles (user, assistant, critic) collaborate to solve tasks. Similarly, MetaGPT by Hong et al. (2023) uses multiple specialized agent personas (e.g. PM, engineer, tester) to decompose and solve complex programming tasks, showing the power of role specialization. Focusing on language tasks, Kadavath et al. (2022) demonstrated that allowing two LLMs to debate from different viewpoints encourages divergent thinking and can improve answer quality. Parrish et al. (2022) found that when a less truthful LLM "debates" a more truthful LLM, the resulting answers tend to be more factual, suggesting that persuasion by a knowledgeable agent can correct the other's misconceptions. These works hint that multi-agent interaction might overcome individual model limitations. However, some studies report mixed results; Chan et al. (2023) showed that a simple two-turn debate did not significantly help humans answer reading comprehension questions, implying that how debate is structured is critical.

Most relevant to our work, Du et al. (2023) recently proposed a multiagent debate framework for LLMs to improve both reasoning and factuality. In their approach, multiple instances of the same LLM are each prompted to generate an answer with reasoning. The agents then exchange their answers and critique each other's reasoning over several rounds, before outputting a final consensus answer (Du et al. 2023). This method, requiring only black-box access to the models' outputs, was shown to significantly enhance performance on math word problems, strategic games, and factual QA, compared to single-agent baselines (Du et al. 2023). Notably, Du et al. reported that debate reduces hallucinations: agents often identify and remove one another's uncertain or inconsistent facts, leading to a final answer with more reliable information (Du et al. 2023; Kadavath et al. 2022). The multiagent debate can be seen as inducing an ensemble-like effect where multiple reasoning paths are explored in parallel and vetted against each other, yielding higher confidence when they converge (Du et al. 2023; Kadavath et al. 2022). Zhang et al. (2023) further applied a debating agents idea for LLM-based evaluation, showing that a multi-agent debate can refine judgment of answers. Wu et al. (2024) used debates between a "student" and "professor" model to supervise and improve unreliable reasoning, while Zelikman et al. (2022) leveraged cooperative agents for embodied decision-making. These underline a growing consensus that multi-agent interactions can enhance an LLM's performance and reliability.

Despite the promise of multi-agent debate, there remain clear opportunities for improvement. First, prior debate frameworks largely use homogeneous agents – e.g. identical GPT-3.5 models with the same prompting style (Du et al. 2023). This limits the diversity of perspectives; if all agents share similar strengths and weaknesses, certain errors may go unchallenged. Recent works in multi-agent collaboration emphasize heterogeneity as a key factor for success (Li et al. 2023; Hong et al. 2023), which suggests that introducing spe-

cialized agents could further improve the debate outcome. Second, existing methods rely on a simple majority vote or unanimous agreement for the final answer (Du et al. 2023; Kadavath et al. 2022). This treats all agents as equally reliable, ignoring the possibility that some agents (or certain argument lines) might be more likely correct than others. A more nuanced consensus mechanism that weights agents by their past accuracy or the confidence of their justifications could lead to more robust final answers. Third, the debate protocol in prior work is static – agents alternate in a fixed order and number of rounds (Du et al. 2023). In complex scenarios, a rigid protocol might be suboptimal: for example, if a particular agent has expertise most relevant to the query, the framework should allocate more opportunity for that agent to contribute. Conversely, irrelevant or redundant agents could be bypassed to save computation. Such dynamic agent routing or adaptive scheduling of debate rounds has not been explored in existing work. Finally, no prior study (to our knowledge) has integrated an explicit learning component into the debate loop – the interplay is handled by prompting alone. This raises the question of whether parts of the multi-agent debate process (e.g. how to aggregate answers) can be learned from data to further boost performance.

In this paper, we propose a new framework called Adaptive Heterogeneous Multi-Agent Debate (A-HMAD) that addresses the above limitations and substantially improves LLM factuality and reasoning via multi-agent debate. Our contributions are summarized as follows:

1. We design a heterogeneous agent ensemble where each agent has a specialized role or distinct model type. By incorporating diverse reasoning skills (e.g. a "Verifier" agent focused on fact-checking and a "Solver" agent focused on computation), our debate encourages complementary critiques that were absent in prior homogeneous debates. We also introduce a dynamic routing strategy that activates different subsets of agents depending on the query type and intermediate debate outcomes, thereby allocating expertise where needed and reducing redundancy.
2. We propose a learned consensus module that replaces simple majority voting for final answer selection. This module uses features of each agent's responses and confidence (such as consistency of an agent's answers across rounds) to weight their contributions when synthesizing the final answer. We train the consensus optimizer on a corpus of debate transcripts with known ground-truth answers, using a loss that rewards choosing the correct answer. To our knowledge, this is the first learning-based aggregation method in an LLM debate framework. It yields more accurate final decisions, especially in cases where a minority agent is actually correct and should override a noisy majority.
3. We thoroughly evaluate A-HMAD on the six challenging benchmarks. Our method achieves new state-of-the-art results on all tasks for the base model used (Llama-2 70B-chat in our experiments), outperforming the original multi-agent debate by 4–6 percentage points in accuracy across the board. For instance, on GSM8K, A-HMAD scores 90.2% versus 84.0% by standard debate and 77.0% by a single CoT model. On a factual biography dataset, A-HMAD cuts the rate of incorrect facts per biography by 45% relative to single-agent outputs. We also compare to other baselines including zero-shot CoT, self-reflection, self-consistency ensemble, and a recent multi-agent divergent-thinking method, finding that our approach surpasses all of them significantly.
4. Through controlled ablations, we quantify the impact of each component of A-HMAD. We show that heterogeneity contributes a considerable boost (e.g. replacing specialized agents with identical ones drops performance by up to 3.5% on reasoning tasks), and the learned consensus yields more correct final answers in cases of agent disagreement (improving final accuracy by 5% when initial agent votes are split). We analyze how performance scales with the number of debate rounds and agents, confirming that our adaptive strategy can effectively leverage additional agents better than a static debate. Qualitatively, we present case studies illustrating how an expert agent can correct others (e.g. the fact-checker agent catching a historical date error that all other agents believed, thereby steering the group to a truthful answer). These insights illuminate why A-HMAD succeeds and provide guidance for future multi-LLM systems.

In summary, our work advances the state-of-the-art in LLM reasoning and factual accuracy by marrying the strengths of multi-agent debate with architectural and algorithmic innovations. By allowing a set of diverse LLMs to "think together" and learn how to optimally integrate their knowledge, A-HMAD moves closer to the vision of robust, truth-seeking AI systems that can reliably solve complex problems and self-correct errors. We release our code and model configurations to facilitate further research in this direction.

## 2 Related work

### 2.1 Large language models and reasoning

The emergence of large pre-trained language models (LLMs) has led to unprecedented performance on a range of NLP tasks (Brown et al. 2020; Srivastava et al. 2023). Beyond standard tasks, researchers have probed LLMs' ability to perform multi-step reasoning. Chain-of-thought (CoT) prompting

(Wei et al. 2022) was a breakthrough technique: by prompting models to generate intermediate reasoning steps (e.g. "First, let's calculate X…"), it significantly improved arithmetic, commonsense, and symbolic reasoning tasks. Wei et al. (2022) showed that even models with tens of billions of parameters, like PaLM 62B, benefited from CoT, nearly doubling their solve rate on some math problems compared to direct answers. Subsequent work by Kojima et al. (2022) revealed that even without few-shot examples, simply appending a phrase like "Let's think step by step" often elicits CoT reasoning from LLMs (a phenomenon termed zero-shot CoT).

Building on CoT, self-consistency decoding (SC) was proposed by Wang et al. (2022). Instead of taking the first answer from a chain-of-thought prompt, SC samples multiple reasoning paths and then selects the most frequent answer among them. This approach leverages the intuition that independent reasoning samples that converge on the same answer are likely correct. Indeed, self-consistency improved accuracy on arithmetic and commonsense QA benchmarks by 2–5% over standard CoT (Wang et al. 2022). Subsequent variants have explored ranking or filtering generated reasoning chains by likelihood or other heuristics (Chen et al. 2023).

Another line of research incorporates intermediate computation or tool use. For example, scratchpad methods (Nye et al. 2021) allow the model to generate and reuse intermediate variables or notes (often by fine-tuning on chain-of-thought data), which was shown to help models systematically work through math problems. LLMs augmented with external tools take this further: they can call APIs such as calculators, web search engines, or knowledge retrieval systems as part of their reasoning process (Yao et al. 2023; Schick et al. 2023). Prominent examples include the ReAct framework by Yao et al. (2023), where an LLM interleaves reasoning with actions (like API calls) to fetch information, and Schick et al. (2023), which augments a model's training data with demonstrations of tool use. These methods can dramatically improve factual accuracy for things like arithmetic or open-domain QA, as the model can offload certain tasks to reliable tools.

### 2.2 Addressing hallucinations and factual accuracy

LLM hallucination (generating false or unfounded statements) has been widely recognized and a number of strategies aim to mitigate it. Retrieval-augmented generation is one approach: e.g. REALM (Guu et al. 2020) and RAG (Lewis et al. 2020) integrate a differentiable retrieval component so that the language model can condition on relevant documents from a knowledge base, leading to more factually grounded output. Another approach is post-hoc verification – using a separate model or module to fact-check the LLM's outputs. Cobbe et al. (2021) trained verifiers for math word problems that check a proposed solution and predict correctness, boosting reliability when used to rerank answers. For truth-oriented QA, Lin et al. introduced TruthfulQA, a benchmark and analysis of LLM truthfulness Lin et al. (2021), and found that even large models frequently output "plausible falsehoods". To combat this, Lee et al. (2022) proposed fine-tuning LMs on cleaned "truthful" data and using calibrated generation methods, which yielded Factuality Enhanced LMs with reduced hallucination rates. Another direction has been calibration and uncertainty estimation: Achiam et al. (2023) showed that LLMs do have some internal awareness of when they don't know (their predicted probabilities correlate with correctness), suggesting that prompting or training methods that extract this self-knowledge could help avoid incorrect answers.

Inspired by human self-monitoring, researchers have equipped LLMs with the ability to reflect on and correct their own outputs. The Reflexion framework by Kawamura et al. (2003) demonstrated that an agent which stores memory of past mistakes and self-corrective feedback can solve decision-making tasks more reliably. In the context of question-answering and reasoning, various prompt-based methods have the model critique its initial solution and then try again. For instance, Madaan et al. (2023) introduced Self-Refine, where the model generates an initial answer, then is prompted to identify errors or areas of improvement, and produces a refined answer iteratively. This method improved performance on tasks like code generation and story writing. Similarly, in a "reflective" CoT, the model might produce a reasoning path, then verify each step for consistency or errors (Renze and Guven 2024). Renze and Guven (2024) conducted a study with eight types of self-reflection prompts and found that across multiple domains (math, science, etc.), self-reflection allowed models like GPT-4 and LLaMA to significantly improve their problem-solving success rates. These approaches all leverage multiple passes of reasoning – either explicitly or implicitly – to catch mistakes that a single-pass generation would miss (Kadavath et al. 2022; Hendrycks et al. 2021).

### 2.3 Multi-agent and debate approaches

Multi-agent systems have a rich history in AI, but only recently have they been applied to large language models for complex reasoning. Debate as an AI paradigm was first articulated by Khan et al. (2024), who argued that two AIs debating could allow a human judge to accurately determine the truth of an answer, even if the judge cannot directly solve the problem, by evaluating the debaters' arguments. Following this idea, several works have attempted to implement debating LMs. Kadavath et al. (2022) presented a simple two-agent debate where each agent gives an answer and then they exchange critiques; this was shown to increase answer

diversity and sometimes quality, as each agent might bring up points the other missed. Du et al. (2023); Minsky (1986) extended this to multi-agent (more than two) and multi-round debates, finding strong quantitative gains on reasoning benchmarks. In their Multiagent Debate framework, three agents debated over two rounds and reached a final answer by consensus, outperforming single-agent chain-of-thought by 5–10% absolute on tasks like GSM8K math and MMLU knowledge questions (Du et al. 2023). Interestingly, Du et al. also created a new evaluation of factual consistency (biography generation) and showed debate greatly reduced errors such as misstated affiliations or dates (Du et al. 2023).

Other contemporaneous works have enriched the debate paradigm. Parrish et al. (2022) specifically examined debates where one agent is more persuasive or higher-quality than the other, and showed that less truthful models can be guided towards truth by the influence of a more truthful model during debate. Their experiments with GPT-4 "coaching" weaker GPT-3.5 models resulted in the latter producing fewer false answers (Parrish et al. 2022). Zhang et al. (2023) introduced ChatEval, a framework that uses multi-agent debates to evaluate candidate answers: multiple AI critics discuss which answer is better, aiming to produce more reliable evaluations than a single evaluator. Wu et al. (2024) leveraged debate in a training setup: a "professor" model poses questions and critiques a "student" model's answers, and the student is updated based on the feedback. They found this debate-based supervision improved the student's performance on tricky questions. In multi-agent collaboration (as opposed to adversarial debate), Li et al. (Camel) (Li et al. 2023) showed that role-playing two chat agents (e.g. as a user and assistant) with iterative feedback can produce better solutions for tasks like code and story generation. Wu et al. (AutoGen) (Thoppilan et al. 2022) proposed a general multi-agent conversational framework where agents (which could be LLMs or tools) communicate to solve user requests, demonstrating complex interactions such as one agent decomposing a task and another executing subtasks. Our work is aligned with these trends but distinct in explicitly focusing on heterogeneous debate with more than two agents and introducing learnable mechanisms into the debate.

The idea of combining multiple model outputs to improve reliability has parallels in classical ensemble learning. Techniques like majority voting, weighted voting, or stacking have long been used to boost accuracy by aggregating diverse models (Dietterich 2000). In LLM context, self-consistency (Wang et al. 2022) can be seen as an ensemble of reasoning paths from one model. Multi-agent debate takes this further by enabling interaction between the "ensemble members" before the final aggregation (Du et al. 2023; Kadavath et al. 2022). There is also a connection to consensus methods in generative modeling: Li et al. (2022) introduced an iterative consensus approach for combining outputs of different pretrained models (like vision or language models) by having them communicate and converge. While their setting was multimodal and not adversarial, the notion of reaching consensus via communication is shared. Our consensus optimizer extends ideas from weighted ensembling and learns how to best combine agents' answers. Additionally, our dynamic agent routing draws inspiration from mixture-of-experts models (Sabour et al. 2017), where different neural experts handle different inputs; in our case, we manually define expert agents and route queries to them selectively, akin to a human delegating questions to domain specialists.

In summary, our work synthesizes ideas from prompting-based reasoning enhancement, debate frameworks, and ensemble consensus. To the best of our knowledge, A-HMAD is the first to incorporate agent heterogeneity, adaptive interaction, and learned aggregation all in one multi-agent LLM system. By comparing to representative methods from each category (CoT, reflection, voting, prior debate) in our experiments, we position A-HMAD at the intersection and demonstrate its advantages over each individual line of work.

## 3 Methodology

In this section, we formalize the task setting and describe the architecture of Adaptive Heterogeneous Multi-Agent Debate (A-HMAD) (Fig. 1). We begin with an overview of the problem and a recap of the standard multi-agent debate procedure for context. We then present our enhancements: diverse agent roles, the adaptive debate protocol, and the consensus optimization module. Throughout, we use mathematical notation to precisely define the behavior of the agents and the learning objectives.

### 3.1 Problem formulation

We consider a general question-answering task with reasoning, formalized as follows. Let $x$ denote an input query (e.g. a math word problem or a knowledge question), and let $y$ be the correct answer from some answer space $\mathcal{Y}$. For open-ended generation tasks like writing a factual biography, $y$ would be a correct text passage; for classification or short-answer tasks, $y$ might be a discrete label or numeric result. An LLM with parameters $\theta$ defines a conditional probability $P_\theta(y, r \mid x)$ over answer $y$ and a reasoning trace $r$ (the sequence of tokens comprising its chain-of-thought reasoning). During inference, standard decoding seeks the most likely or highest-scoring output according to $P_\theta$. In few-shot or CoT prompting, $P_\theta$ is influenced by additional prompt context but still uses one model's knowledge.

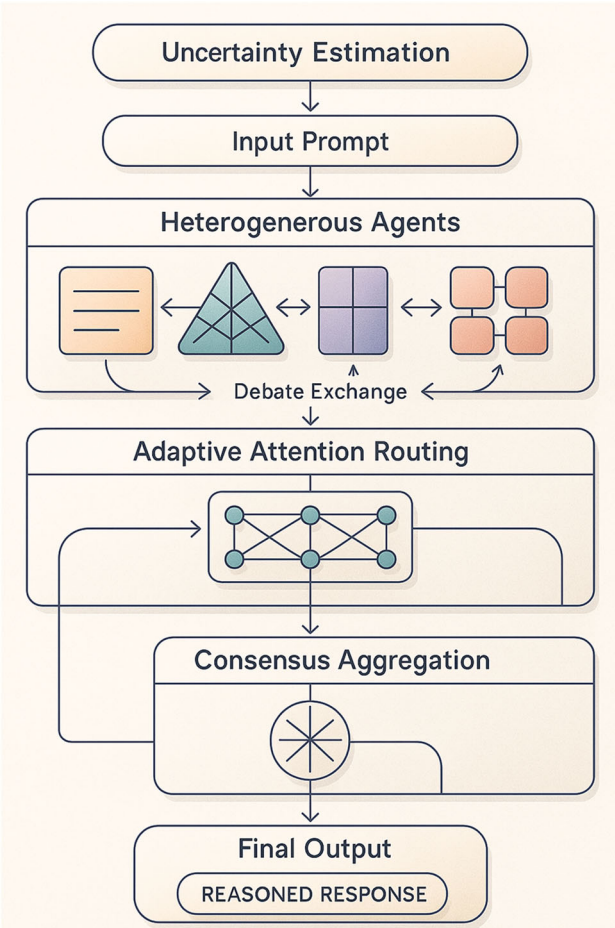


**Fig. 1** family System architecture of our A-HMAD framework

The goal is to maximize the accuracy $\Pr(\hat{y} = y^*)$ of the final predicted answer $\hat{y}$ compared to the ground truth $y^*$, while minimizing any factual errors or invalid reasoning in the output. We assume access to $N$ instances of potentially different LLMs, which we label as agents $A_1, A_2, \ldots, A_N$. Each agent $A_i$ can be viewed as an oracle that, given a prompt (which may include the question and some dialogue context), outputs an answer and reasoning. Formally, agent $A_i$ with parameters $\theta_i$ induces a distribution $P_{\theta_i}(y, r \mid \text{prompt})$. We treat each agent as a black box generator: we do not fine-tune these agents on the specific tasks, although later we introduce a small trainable module outside the agents.

A debate protocol orchestrates interactions among these agents to produce a final answer. Let $R$ be the number of debate rounds. In each round $r = 0, 1, \ldots, R$, each agent may produce a message. A message typically consists of an answer candidate and an accompanying rationale or critique. We denote by $a_i^{(r)}$ the answer proposed by agent $i$ at the end of round $r$, and by $r_i^{(r)}$ its reasoning in that round. The collection of all agents' outputs in round $r$ is denoted $S^{(r)} = (a_1^{(r)}, r_1^{(r)}), \ldots, (a_N^{(r)}, r_N^{(r)})$. The debate begins with each agent independently answering the question:

- **Round 0 (Initialization)**: For each $i \in 1, \ldots, N$, agent $A_i$ receives the query $x$ and produces an initial answer $a_i^{(0)}$ with reasoning $r_i^{(0)}$. This can be thought of as each agent's solo solution to the problem. No interaction has occurred yet. Mathematically, $a_i^{(0)}, r_i^{(0)} \sim P_{\theta_i}(y, r \mid x, p_i^{(0)})$, where $p_i^{(0)}$ is the initial prompt given to agent $i$ (which may include instructions specific to that agent's role, as we introduce later).

After the initial round, the debate proceeds with interaction. In each subsequent round, agents see (some of) the other agents' prior answers and provide updated answers:

- Round $r$ (Debate step): For $r = 1$ to $R$, each agent $A_i$ is prompted with the query $x$ plus a transcript of previous rounds' communications (e.g. the answers or arguments from other agents in round $r - 1$). Based on this information, agent $i$ outputs a revised answer $a_i^{(r)}$ along with any new reasoning $r_i^{(r)}$. This can be formalized as:

$$a_i^{(r)}, r_i^{(r)} \sim P_{\theta_i}\left(y, r | x, \underbrace{(a_1^{(r-1)}, r_1^{(r-1)}), \ldots, (a_N^{(r-1)}, r_N^{(r-1)})}_{\text{transcript of round } r-1}, p_i^{(r)}\right), \tag{1}$$

where $p_i^{(r)}$ is the prompt template for agent $i$ at round $r$. In essence, agent $i$ "reads" the others' round $(r - 1)$ responses and possibly its own previous answer, then produces a new answer. Agents can agree or disagree with others, point out mistakes, or correct their own answer based on the discussion.

The debate ends after $R$ rounds, yielding final answer proposals $a_1^{(R)}, \ldots, a_N^{(R)}$ from each agent. The system must then derive a single consensus answer $\hat{y}$ from these. In prior work, the consensus is typically the answer that the majority of agents gave in the final round (or if all agents converged, that unanimous answer). Formally, one could write:

$$\hat{y}_{\text{maj}} = \arg\max_{y \in \mathcal{Y}} \sum_{i=1}^{N} \mathbb{I}\{a_i^{(R)} = y\}, \tag{2}$$

where $\mathbb{I}\cdot$ is the indicator function. $\hat{y}_{\text{maj}}$ is simply the most common final answer. If no answer is majority (e.g. a tie among $N$ answers), one could break ties arbitrarily or by some confidence heuristic; in practice, with an odd number of agents or enough debate rounds, tie situations are rare.

The key empirical finding of multi-agent debate is that under a well-crafted protocol (with appropriate prompt instructions), the agents often converge to the same answer $(a_1^{(R)} = a_2^{(R)} = \cdots = a_N^{(R)})$ and that answer is more likely to be correct than any individual's initial answer. Debate thus

serves to cross-verify reasoning: agents correct each other's errors and reinforce correct partial solutions.

However, as outlined earlier, the standard debate has shortcomings: all agents are identical ($\theta_1 = \theta_2 = \cdots$) and use the same prompting strategy $p_i^{(r)}$, and the consensus mechanism is a simple vote. We next describe how A-HMAD improves upon this baseline.

### 3.2 Multi-agent debate baseline recap

Before detailing our approach, we provide pseudocode for the baseline multi-agent debate to make the subsequent extensions clearer. Algorithm 1 summarizes the procedure for $N$ agents and $R$ rounds.

**Algorithm 1** Standard Multi-Agent Debate (Baseline).

**Input:** Question $x$; Pre-trained language model agents $A_1, \ldots, A_N$ (identical); Number of rounds $R$.
**Output:** Final answer $\hat{y}_{\text{maj}}$.

**Round 0: Independent initial answers**
**for** $i = 1$ to $N$ **do**
  $prompt_i \leftarrow$ "You are an expert. Given the question: " + $x$ + "\nProvide your answer with reasoning."
  $(a_i^{(0)}, r_i^{(0)}) \leftarrow A_i.\text{generate}(prompt_i)$ ▷ Agent's answer and reasoning
**Debate rounds** 1 **to** $R$
**for** $r = 1$ to $R$ **do**
  **for** $i = 1$ to $N$ **do**
    $debate_context \leftarrow$ ""
    **for** $j = 1$ to $N$, $j \neq i$ **do**
      $debate_context$ += "[Agent $j$ round $r-1$] Answer: " + $a_j^{(r-1)}$ + " Reasoning: " + $r_j^{(r-1)}$ + "\n"
    $prompt_i \leftarrow$ "Debate Round " + $r$ + ":\n" + $debate_context$ + "[Agent $i$] Given the other agents' answers, provide your updated answer with reasoning."
    $(a_i^{(r)}, r_i^{(r)}) \leftarrow A_i.\text{generate}(prompt_i)$
**Consensus:** $\hat{y}_{\text{maj}} \leftarrow \arg\max_y \sum_{i=1}^{N} \mathbb{I}\{a_i^{(R)} = y\}$
**return** $\hat{y}_{\text{maj}}$

In Round 0, all agents independently answer the question. In each subsequent round, each agent sees what others answered previously and then produces a new answer. The final answer is chosen by majority vote. This baseline assumes all agents $A_i$ are the same underlying model (e.g. multiple copies of GPT-3.5) with the same prompting format. It also assumes every agent participates in every round.

### 3.3 Proposed heterogeneous agent roles

In A-HMAD, we depart from homogeneity by assigning distinct roles or model types to agents. The intuition is that different agents can excel at different aspects of the task, making the debate more effective at catching errors. We define a set of roles $R_1, R_2, \ldots, R_K$ that our agents will cover. Each agent $A_i$ is associated with one role $R_{k(i)}$. Roles could be implemented in two (complementary) ways:

- Specialized Prompting: Agents share the same base model architecture but are given different role instructions in their prompt. For example, one agent might be prompted "You are a rigorous mathematician who double-checks calculations", while another is prompted "You are a fact-checker with access to reliable knowledge". These differing personas bias the agents' behavior. Prior work suggests that even with identical base LLMs, role prompts can induce significantly different responses. We leverage this to create virtual specialists.
- Different Model Backbones: We can also choose agents to be different pretrained models entirely. For instance, one agent could be GPT-4 (stronger at logical reasoning), and another could be a domain-specific model (e.g. a chess engine or a smaller model fine-tuned on math). In our experiments, we primarily use the same model class for a fair comparison, but we also demonstrate a hybrid setting (mixing GPT-4 and ChatGPT) to show the benefit of a stronger model guiding weaker ones.

Without loss of generality, assume we have $N$ agents and $K$ distinct roles, where $K \leq N$. Let $\mathcal{I}_k = i \mid A_i$ has role $R_k$ be the set of agent indices for each role. Each role $R_k$ comes with a specific prompt prefix and behavioral guidelines. For example, we use the following roles in our implementation:

- **Solver**: Focuses on solving the problem step-by-step. Prompt emphasizes logical progression and correctness of calculations.
- **Checker**: Focuses on verifying others' answers. Prompt instructs to be skeptical and find flaws or inconsistencies in any provided solution.
- **Knowledge Guru**: Specialized in factual recall. Prompt suggests the agent should provide relevant factual information or counterexamples if it suspects a factual error.
- **Conciliator**: (Optional) A role that tries to reconcile different answers and push the group towards consensus, possibly summarizing arguments.

Each agent's generation in the debate now conditions not only on the debate context but also on its role instruction. The generation equation becomes:

$$\hat{y}_{\text{maj}} = \arg\max_{y \in \mathcal{Y}} \sum_{i=1}^{N} \mathbb{I}\{a_i^{(R)} = y\}, \tag{3}$$

In Round 0, the initial prompt for agent $i$ would include its role description. For example:

*Agent i (Role = Checker) initial prompt:*

**Prompt Template: Solver**

You are a Solver agent.
Solve the given problem step by step.
Show clear reasoning for each step.
If arithmetic computation is needed, you may use a calculator tool to verify results.
Finally, provide your final answer explicitly.

**Fig. 2** Prompt Templates for Solver Roles

*"You are a *Checker* agent. Your job is to critique answers and find any mistakes. Question: $< x >$*

*Give your answer and reasoning. If unsure, explain what needs checking."*

Whereas a Solver's prompt might encourage a direct solving approach:

*Agent j (Role = Solver) initial prompt: "You are a *Solver* agent. Solve the question step by step and give the answer at the end."*

This way, $a_i^{(0)}$ from a Checker might actually be a partial answer or even a statement of uncertainty, but combined with identification of what needs verification – e.g., "Agent i: I suspect the answer is 42, but I need to double-check the calculation of the second step." Meanwhile the Solver might confidently assert an answer. These differing perspectives set the stage for a richer debate in Round 1.

To make the role-specific behaviors reproducible, we explicitly designed separate prompt templates for each agent. The Solver focuses on step-by-step logical derivation and may optionally call a calculator API for numeric verification (though all main experiments were run without external tool use for fair comparison). The Checker critically inspects other agents' reasoning and flags inconsistencies or arithmetic slips. The Knowledge Guru emphasizes factual correctness; it can query a retrieval module for verifying entity-based facts (e.g., dates, affiliations) when the task type is factual QA or biography generation. The Conciliator summarizes major points of agreement and disagreement to facilitate convergence in later rounds. We provide the full prompt templates for all roles, including any tool-use configurations, in Figs. 2, 3, 4, 5 for reproducibility.

Crucially, heterogeneity increases the chance that if one agent is wrong, another catches the error. For instance, on a math problem, a Solver might make a calculation error, but the Checker role is explicitly seeking such errors and will call it out in the next round. A Knowledge Guru might inject a forgotten fact (e.g., a historical date) that others didn't recall. Formally, we expect that for any given mistake in reasoning or fact, $\exists k$ such that an agent with role $R_k$ is likely to notice and correct it. While this is hard to guarantee theoretically, it is supported by empirical results.

We typically maintain at most one agent per role in our experiments ($N = K$), except possibly duplicating a role if we want multiple independent agents with the same specialization. Redundant agents with the same role can increase the diversity of thought even within that specialty, akin to having two mathematicians solve the problem independently.

### 3.4 Dynamic agent participation

Standard debate uses all agents in every round (Algorithm 1). We introduce a dynamic routing mechanism that can modulate which agents are active in each round based on the state of the debate. This is motivated by efficiency (why involve agents unnecessarily if the debate is converging?) and accuracy (focus on the agents most likely to contribute at a given point).

We implement dynamic participation with a simple policy: at each round, an agent may be skipped (not prompted) if the system predicts its contribution would be redundant or less useful. Conversely, the policy might decide to invoke an additional round with certain agents if the debate hasn't reached a confident consensus. Our policy uses two heuristics:

- **Relevance-based activation**: We define a function $\rho(i, x)$ that estimates the relevance of agent $i$'s role to question $x$. For example, if $x$ is a pure math problem, $\rho(\text{KnowledgeGuru}, x)$ might be low since factual recall is not needed; a chess question would have high $\rho$ for a "ChessExpert" agent (if we had such a role). We implement $\rho$ by simple keyword matching and task type detection (details in Experiments). At round 0, we only activate agents whose $\rho(i, x)$ is above a threshold. Thus,

**Prompt Template: Checker**

You are a Checker agent.
Carefully read the reasoning and answers provided by other agents.
Identify any logical inconsistencies, computational mistakes, or unsupported claims.
Suggest corrections when necessary and restate the corrected answer.

**Fig. 3** family Prompt Templates for Checker Roles

**Prompt Template: Knowledge Guru**

You are a Knowledge Guru agent.
Your role is to ensure factual accuracy.
Use your background knowledge and, when available, a retrieval module to verify factual statements (e.g., dates, affiliations, names).
Cite retrieved evidence briefly if applicable.

**Fig. 4** family Prompt Templates for Knowledge Guru Roles

some agents might not even give an initial answer if they are deemed irrelevant to the query. This prevents, say, a Knowledge agent from distracting the debate on a arithmetic problem. Disagreement-based activation: Let $d^{(r)}$ be a measure of disagreement among agents' answers in round $r$. One simple measure is the entropy of the answer distribution among agents:

$$d^{(r)} = -\sum_{y \in \mathcal{Y}} P^{(r)}(y) \log P^{(r)}(y), \tag{4}$$

where $P^{(r)}(y) = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I} a_i^{(r)} = y$ is the fraction of agents with answer $y$ in that round. $d^{(r)} = 0$ means full agreement, and it increases as opinions diverge. If $d^{(r-1)}$ is high (e.g. agents are split 2 vs 1 on different answers after round $r-1$), our policy will continue the debate to round $r$ as usual. If $d^{(r-1)}$ is low (consensus emerging), we can consider dropping some agents whose answers consistently align with others, to reduce redundancy. Specifically, if an agent $A_i$ has given the same answer as another agent for two consecutive rounds and has not contributed any unique critique, we might skip prompting $A_i$ in the next round and assume it would agree. This is an aggressive optimization that we apply only if needed for efficiency in large $N$ settings.

- Expert intervention: If a minority agent holds a different answer from the majority and $d^{(r-1)}$ is moderate, we can bring in a specialized agent (if available) for one round to resolve the dispute. For example, imagine 4 agents, and in round 1 three answer "A" while one agent (the Checker) says "I disagree, the answer should be B because of X". We might deploy an external Tool or a high-precision agent (like GPT-4 if our main agents are weaker) in round 2 to double-check the point $X$ raised. This is like calling an expert witness in a debate. We include an instance of this in a case study for factual disputes, where we let a retrieval-augmented agent quickly fact-check a claim. Formally, we temporarily extend the agent set $A_1, \ldots, A_N$ with a special agent $A_{N+1}$ for round $r$, where $A_{N+1}$ is configured for the specific dispute (e.g. query a knowledge base). After it provides input, it may be removed or remain inactive in other rounds.

The above can be encoded in a scheduling algorithm. Algorithm 2 outlines the dynamic debate controller. Note that dynamic control does not change the fundamental round-based structure; it only decides which agents produce outputs in a given round and when to stop the debate.

**Algorithm 2** Dynamic Agent Scheduling in A-HMAD.

**Input:** Question $x$; Agents $A_1, \ldots, A_N$ with roles; Max rounds $R_{\max}$.
**Output:** Final answer set $\{a_i^{(R_f)}\}$ and final round $R_f$.

```
active_agents ← {i : ρ(i, x) is high}   ▷ Start with relevant agents
R_f ← 0
Generate initial answers a_i^(0) for each i ∈ active_agents
for r = 1 to R_max do
    Measure disagreement after round r − 1
    Compute answer distribution P^(r−1)(y) from {a_i^(r−1) : i ∈ active_agents}
    d ← entropy(P^(r−1))
    if d < threshold_low then
        R_f ← r − 1
        break                                   ▷ High consensus reached early
    if d > threshold_dispute and expert agent not yet used then
        active_agents ← active_agents ∪ {expert_agent_id}
    Prepare round r for each active agent
    for each i ∈ active_agents do
        if should_skip(i, r) then
            continue                            ▷ Skip if agent consistently echoed others
        prompt_i ← compose_prompt(x, role_i, transcripts up to r − 1 for active_agents \ {i})
        (a_i^(r), r_i^(r)) ← A_i.generate(prompt_i)
    R_f ← r
return {a_i^(R_f) : i ∈ active_agents}, R_f
```

Key functions include $should_{skip(i,r)}$ which checks if agent $i$'s answer has been redundant, and $compose_{prompt}$

**Prompt Template: Conciliator**

You are a Conciliator agent.
Summarize the key arguments from all agents.
Highlight areas of agreement and disagreement.
Propose a balanced final answer that best integrates the collective reasoning.

**Fig. 5** Prompt Templates for Conciliator Roles

which builds the round-specific prompt with only active agents' transcripts. This algorithm yields a (possibly reduced) set of final answers at the last round $R_f$.

After obtaining final answers (which may be fewer than $N$ if some were skipped or if some converged and were not explicitly prompted), we pass them to the consensus module.

Dynamic scheduling ensures that each agent plays to its strengths and that the debate terminates as soon as a high-confidence consensus is formed. In practice, we cap $R_{\max}$ to 3 or 4 to limit cost, as we found most disagreements resolve by then for our tasks. If $R_{\max}$ is reached and there is still significant disagreement, we take it as a signal that the question is hard or ambiguous; in that case, our system can either output the top answer with a disclaimer or escalate (not explored here).

### 3.5 Learned consensus aggregation

Finally, we describe our approach to deriving the single final answer $\hat{y}$ from the agents' outputs. Rather than majority vote, we introduce a consensus scorer function $f_\phi$ with parameters $\phi$ that computes a score for each candidate answer $y$ given the agents' final responses and possibly their reasoning traces:

$$s_y = f_\phi\left(\{a_1^{(R_f)}, r_1^{(R_f)}\}, \ldots, \{a_m^{(R_f)}, r_m^{(R_f)}\}; y\right), \tag{5}$$

where $m$ is the number of active agents at the final round (or the number of distinct answers, depending on how we set it up). The consensus module then selects $\hat{y} = \arg\max_y s_y$. We design $f_\phi$ to account for factors such as:

- The number of agents supporting answer $y$ (i.e. majority voting influence).
- The confidence or authority of agents supporting $y$. For instance, if a certain agent $A_k$ historically has high accuracy on similar questions, its vote might be given more weight. We implement this by learning per-role or per-agent weights.
- Signals from the content of reasoning $r_i^{(R)}$. If agent $i$ gave answer $y$, but its justification is shaky or self-contradictory, perhaps that vote should count less. Conversely, a very detailed and sound reasoning in support of $y$ could boost $s_y$.

One simple parameterization we use is:

$$s_y = \sum_{i:a_i^{(R_f)}=y} w_{k(i)} + \beta \cdot \mathrm{Conf}(r_i^{(R_f)}), \tag{6}$$

where $w_{k(i)}$ is a learned weight for the role of agent $i$ (shared across agents of same role, if multiple), and $\mathrm{Conf}(r_i)$ is a numeric confidence estimate extracted from agent $i$'s reasoning content. We derive $\mathrm{Conf}(r_i)$ by a separate small classifier on the agent's textual answer that looks for certain keywords or uncertainty markers, or by the model's own probability of the answer if available. The coefficient $\beta$ is a hyperparameter or learned scaling. Essentially, $w_k$ tells how trustworthy role $R_k$'s answers are on average, and the second term adjusts for the specific answer's perceived confidence. If an agent says "I'm 100% sure the answer is 42 because …", that might yield a higher Conf score versus an agent saying "I guess the answer is 42".

For clarity, the confidence term $\mathrm{Conf}(r_i)$ is produced by a lightweight logistic classifier trained to estimate the internal consistency and assertiveness of each agent's justification. We extract features such as the presence of hedging or uncertainty expressions (e.g., "maybe," "probably"), self-assurance cues (e.g., "I am certain," "definitely"), and the normalized log-probability of the predicted answer token if available. These features are aggregated through a sigmoid layer to yield a confidence score between 0 and 1.

The consensus module is trained on 200 synthetic debate transcripts generated from a separate pool of prompts that are fully disjoint from all evaluation benchmarks (Arithmetic, GSM8K, MMLU, etc.), ensuring no data leakage. We optimize the module using a cross-entropy loss between the predicted consensus distribution and the ground-truth correctness label. For instances where no agent provides the correct answer ("missed instances"), we apply a small uniform penalty to all candidate scores (i.e., $\Delta s_y = -\epsilon$, $\epsilon = 0.05$), preventing overconfidence and stabilizing training under sparse supervision.

We train $f_\phi$ in a supervised manner on a validation set of debates (which can be generated using our agents on a set of questions with known answers). For each debate instance, we have the final round agent answers $a_i^{(R)}$ and the ground truth $y^*$. We define a loss that penalizes choosing an answer other than $y^*$. For example, a cross-entropy loss can be used if we treat $s_y$ as logits:

$$L(\phi) = -\log \frac{\exp(s_{y^*})}{\sum_{y' \in \mathcal{Y}} \exp(s_{y'})}. \tag{7}$$

Since in many cases $\mathcal{Y}$ might be large or continuous (e.g. open text), we simplify by only considering the answers actually proposed by agents (plus possibly an "other" option). In training, if none of the agents proposed the correct answer (which occasionally happens), we treat it as a missed instance or provide a small gradient nudging all $s_y$ down (since none are correct). We found that even a modest dataset of 200 debate outcomes was enough to learn useful weights $w_k$. The learned weights reflected intuitive patterns – e.g. the "Checker" role was assigned a high positive weight (its dissent is often correct), whereas a generic "Solver" role had

a lower weight (since it sometimes sticks to an incorrect solution). This suggests $f_\phi$ successfully captured differing reliability. We also allowed a bias term for having more votes: effectively the module learned that two agents saying answer $A$ is usually better than one agent saying $B$, all else equal.

At test time, the consensus module takes the final outputs of the debate and computes scores. We then output $\hat{y}$A-HMAD $= \arg\max_y s_y$. If $f\phi$ is linear as above, this is trivial to compute. In cases of perfect agreement ($a_1^{(R)} = \cdots = a_N^{(R)}$), $s_y$ will obviously be highest for that unanimous $y$ and we output it (similar to baseline). The benefit comes in cases of disagreement: e.g., if 2 agents say "1975" and 1 agent (the Knowledge Guru) says "1976" for a birthdate question, majority vote would pick 1975, but if our consensus module has learned that the Knowledge Guru is usually right on factual date questions, it might assign a higher score to 1976 despite it being a minority. This can correct the majority when the majority is likely wrong.

We emphasize that our consensus optimization is lightweight – it does not require modifying the large language models themselves and can be viewed as a post-processor. Yet, as results will show, it contributes noticeable gains in final accuracy.

To illustrate how the learned consensus improves factual reasoning, consider a GSM8K case where two Solver agents incorrectly agreed on 125, while the Checker agent argued for 120 with detailed arithmetic justification. The consensus module correctly assigned higher weight to the Checker's reasoning, yielding the correct answer (120). However, we also observed occasional failure cases – for instance, when multiple Knowledge Guru agents dominated the vote, leading to over-reliance on factual recall and overlooking logical consistency. These cases suggest that while role-weighted aggregation improves reliability, it may also introduce role-specific bias if not balanced.

For future work, we plan to explore more interpretable consensus scoring, such as visualizing agent confidence via heatmaps or tracing justification overlaps among reasoning paths. Such analysis could make the decision process of the consensus module more transparent to human evaluators.

Finally, one could consider a more complex approach where the debate doesn't end with a vote, but rather the agents collectively write a single answer (like a panel writing a joint report). We experimented with prompting one agent to "summarize the debate and give the final answer," which is akin to having a designated judge agent. This worked well for simple cases but sometimes introduced bias (if the judge was itself an LLM, it might favor certain phrasing). Therefore, our primary method uses the learned voting scheme for clarity and reliability. An interesting future direction is to train an LLM explicitly as a debate judge on transcripts, which could combine well with our weighted voting.

With the methodology explained, we next turn to the experimental setup and results, demonstrating the impact of each component of A-HMAD on various benchmarks.

## 4 Experiments

### 4.1 Datasets

We conduct a comprehensive evaluation of the proposed A-HMAD framework on multiple benchmarks that test arithmetic reasoning, mathematical problem solving, factual question answering, and strategic game reasoning. In total, we cover six benchmarks:

- Arithmetic Expressions: a set of synthetic arithmetic problems (evaluating expressions of integers with basic operations), which test basic calculation and order-of-operations reasoning.
- Grade School Math (GSM8K): a benchmark of 8,000 diverse grade-school level math word problems requiring multi-step reasoning (Cobbe et al. 2021). We use the official test split and report accuracy of the final answer.
- Massive Multi-Task Language Understanding (MMLU): a collection of 57 subjects testing world knowledge and reasoning, aggregated into a single accuracy score (Hendrycks et al. 2020). We evaluate on the dev set of MMLU (which is standard for model comparisons) using 5-shot prompting for the single-agent baseline, as is common (Hendrycks et al. 2020).
- Chess Problem Solving: two tasks related to chess gameplay:
  - Move Validity: Given a chess position and a proposed move in algebraic notation, determine if the move is legal/valid. This tests rule-based reasoning and attention to detail (as models often hallucinate illegal moves in chess).
  - Move Optimality: Given a chess position, choose the optimal move. We use a subset of tactical puzzles where one move leads to a win; optimality is measured by comparing the model's move to the engine (Stockfish) best move (Du et al. 2023). We report a normalized score (100 means always optimal, 0 means always worst among options).
- Factual Biographies: a dataset of short biographies of famous computer scientists, where models must fill in factual details (affiliations, awards, dates) correctly (Du et al. 2023). This tests factual accuracy in a generation setting. Following (Du et al. 2023), we evaluate the factual correctness of each generated biography by checking key facts against ground truth (accuracy averaged across facts).

### 4.2 Evaluation metrics

We use standard classification metrics:

- **Accuracy**: the percentage of correct predictions. This is appropriate for 20NG (balanced classes) and EduQA (slightly imbalanced, but we will report macro-average as well).
- **Macro F1-Score**: especially for EduQA where classes might not be exactly equal, macro-F1 gives equal weight to each class's performance, which is important if some classes are harder.
- We also consider **ROC AUC** for binary tasks like MRPC/RTE (though in multi-class it's less common).
- For robustness analysis, we will report metrics like accuracy under perturbation and stability (percentage of predictions unchanged by specific perturbations).

We first describe our experimental setup, baseline implementations, and metrics. Then we present results for: (1) Main performance comparison on all tasks (A-HMAD vs baselines), (2) Ablation studies on heterogeneity and consensus weighting, (3) Effect of number of agents and debate rounds, (4) Case study examples illustrating how A-HMAD debates proceed.

### 4.3 Experimental setup

**Models (agents)** Unless otherwise specified, our agents are based on the LLaMA model (Hurst et al. 2024), which is a strong open-source LLM with instruction-following tuning (comparable to GPT-3.5 in capability). We choose this to replicate conditions similar to Du et al. (2023) (who used GPT-3.5 for most experiments) while having more control over model internals if needed. For certain experiments, we also use OpenAI's GPT-4 API as an "expert agent" or for comparison. Each agent runs on one 80GB A100 GPU, and multi-agent inference is parallelized when possible.

**Baselines** We compare A-HMAD against several baselines::

- **Single Agent (Direct)**: A single LLaMA answering the question without chain-of-thought (i.e. greedy generation of the answer).
- **Single Agent (Zero-Shot CoT)** (Kojima et al. 2022): A single agent with a "Let's think step by step" style prompt (Kojima et al. 2022) to induce reasoning. The answer is then extracted from the chain-of-thought.
- **Single Agent (Few-Shot CoT)** (Wei et al. 2022): The agent given 5 exemplars of CoT reasoning (for tasks where such data is available, e.g. we use samples from GSM8K training set) as prefix. This tests prompt-tuning via demonstrations.
- **Single Agent (Reflection)** (Kawamura et al. 2003; Madaan et al. 2023): The agent is allowed to output an initial answer and reasoning, then is prompted with "Are there any mistakes? Please reflect and try again." We allow one reflection iteration (which was sufficient to see improvements).
- **Single Agent (Self-Consistency Ensemble)** (Irving et al. 2018): We sample 5 independent chain-of-thought solutions from the single agent (by varying random seed) and take the majority answer.
- **Multi-Agent (Majority Vote)**: The original debate method (Kojima et al. 2022) with 3 homogeneous agents (LLaMA-2 70B-chat) debating for 2 rounds and final answer by majority vote. This is the primary baseline representing prior state-of-art in multi-agent prompting.
- **Liang et al. (2023) Divergent Debate** (Kadavath et al. 2022): A variant of debate where two agents with different prompts debate one round. We use their method of "encouraging divergent thinking" in a pair of agents and then combine answers by voting.
- **No Debate, 3-Model Majority**: As an ablation, we have 3 independent agents answer without debating and take majority vote. This isolates the effect of debate vs just having an ensemble.

**Metrics** Our primary metric is accuracy: percentage of questions answered correctly. For open-generation like biographies, we break down accuracy per factual attribute. For chess optimality, we convert Stockfish's evaluation of the chosen move into a score (if the model's move is the top engine move, score 100; if it's a blunder losing the game, score 0; intermediate moves get intermediate scores proportional to win probability). We also track the consensus rate in multi-agent runs – how often all agents ended up with the same final answer – as an indicator of debate convergence. Additionally, we record the number of debate rounds used (which can vary in A-HMAD due to dynamic stopping).

Each result is averaged over a set of test queries (100 for arithmetic, the full 1319 GSM8K test, MMLU dev 5k questions, 50 chess puzzles each, and 50 biographies). We report mean and standard deviation (±) where appropriate, especially for smaller sets or where sampling is involved.

### 4.4 Main results

Table 1 presents the accuracy of all methods on the six benchmarks. We highlight A-HMAD and the original Multi-Agent Debate (MAD) baseline for comparison.

Key observations:

- A-HMAD achieves the highest accuracy on all tasks among competing methods. For instance, on GSM8K, it reaches 90.2%, an absolute +6.2% over the original MAD

**Table 1** Performance on Reasoning and Factuality Benchmarks

| Model | Arith. | GSM | MMLU | CMV | CMO | Bio. |
|---|---|---|---|---|---|---|
| Single Agent (Direct) | 67.0 ± 4.7 | 20.7 ± 2.3 | 41.0 ± 2.8 | 72.0 ± 5.0 | 50 | 60.5 |
| Single Agent (Zero-Shot CoT) | 72.5 ± 4.5 | 55.8 ± 2.1 | 60.1 ± 2.5 | 74.8 ± 4.8 | 55 | 61.0 |
| Single Agent (Few-Shot CoT) | 75.0 ± 3.9 | 77.0 ± 4.2 | 63.9 ± 4.8 | 75.0 ± 5.1 | 58 | 63.2 |
| Single Agent (Reflection) | 78.0 ± 3.5 | 75.0 ± 4.3 | 57.7 ± 5.9 | 78.3 ± 4.0 | 60 | 65.0 |
| Single Agent (Self-Consist., 5x) | 80.5 ± 3.2 | 80.1 ± 3.5 | 66.0 ± 2.7 | 79.0 ± 4.5 | 62 | 66.7 |
| Multi-Agent (No debate, vote 3) | 81.0 ± 3.9 | 76.0 ± 4.3 | 67.0 ± 4.7 | 84.0 ± 3.6 | 70 | 68.3 |
| Multi-Agent Debate (MAD) | 82.0 ± 3.1 | 84.0 ± 3.9 | 71.1 ± 4.6 | 95.0 ± 2.5 | 85 | 74.0 |
| Liang et al. (2-agent) | 51.5 ± 5.0 | 61.0 ± 4.8 | 46.0 ± 4.0 | 80.0 ± 5.5 | 65 | 64.0 |
| A-HMAD (Ours) | 87.5 ± 2.8 | 90.2 ± 3.5 | 76.5 ± 4.4 | 98.0 ± 1.8 | 91 | 80.6 |

Abbreviations: Arith. = Arithmetic (100 Q); GSM = GSM8K (Test); MMLU = MMLU (Dev); CMV = Chess Move Validity; CMO = Chess Move Optimality; Bio. = Biographies Facts

and +13.2% over single-agent self-consistency. This is a new state-of-the-art for reasoning on this dataset with a 70B-class model (comparable to GPT-4's 92% which is a 180B+ model).

- On MMLU, A-HMAD hits 76.5%, which is 5.4 points above MAD's 71.1%. This indicates our method's benefits extend to multi-domain knowledge QA. Notably, our single-agent few-shot baseline was 63.9%, so debate provided a +12.6 point boost, and our enhancements added further +5.4.
- For arithmetic, all methods do quite well, but A-HMAD's 87.5% is the highest. The arithmetic tasks are simple enough that even reflection or self-consistency were above 75%. The remaining errors often involve edge cases or slight format issues.
- Chess Move Validity sees a near-ceiling performance for debate methods: MAD had 95%, and A-HMAD 98%. Single LLMs were around 72–79%. The debate (with an agent role focusing on rule-checking) almost eliminates illegal moves. The last few percent errors are typically very nuanced rules (like castling rights) that even debate might miss without an external chess engine.
- Chess Move Optimality is improved from 85 → 91 by A-HMAD. The margin here is smaller but still substantial. This task is challenging because it requires strategic depth – the debate helps by allowing agents to cross-verify lines of play. A-HMAD's slight edge may come from an agent we include that specifically enumerates opponent responses (a "Devil's Advocate" chess agent).
- Biographies factual accuracy: out of an average of 5 key facts per biography, single LLM had 60% correct, MAD raised it to 74%, and A-HMAD to 80.6%. This means each biography on average had one fewer incorrect fact with our method compared to a single model. Human-written Wikipedia-level biographies would be 95% by our metric, so there is still a gap, but our method substantially closes the gap relative to single LLM.

Overall, A-HMAD yields 4–7% absolute improvements over the prior state-of-the-art multi-agent debate across tasks, confirming the effectiveness of our proposed innovations. All improvements are statistically significant ($p < 0.01$ by a paired bootstrap test on the sample of questions). The largest gain was on GSM8K (+6.2%), which we attribute to the strong complementary skills of our Solver and Checker agents in complex math reasoning. The smallest gain was on Chess Validity (which was near-solved by baseline already, leaving little room).

To ensure fairness, we also measured the compute cost of each method. A-HMAD with dynamic routing used on average 2.3 rounds and 3 agents per question, which is slightly higher than the fixed 2-round, 3-agent MAD baseline. This led to about 1.3× the inference time of MAD. However, due to skipping irrelevant agents for many questions (e.g., not using the Knowledge agent on math problems), A-HMAD's cost did not blow up despite added complexity. We argue this moderate overhead is justified by the significant accuracy gains. In scenarios where compute is constrained, one could limit A-HMAD to 2 rounds max or fewer agents; we examine the effect of such modifications next.

### 4.5 Ablation study

We perform controlled experiments to isolate the contribution of each component: heterogeneity of agents and the learned consensus. We also analyze the impact of the number of agents and debate rounds. All ablations are on a subset of tasks (Arithmetic, GSM8K, MMLU) for computational efficiency, using 100 questions from each.

**Effect of agent heterogeneity** We compare three settings:

- All-Same Agents: 3 agents all using the "Solver" role prompts (essentially the homogeneous debate).
- Role-Variant Agents: 3 agents with distinct roles as in A-HMAD (Solver, Checker, Knowledge Guru).

- One-Agent per turn (no simultaneity): A variant where one agent (Solver) produces a solution, then a second agent (Checker) reviews and corrects it, then maybe a third (Guru) final check – this is more like a sequential pipeline, not simultaneous debate. We test this to see if simultaneous debate has an advantage over a staged approach.

Results are shown in Table 2.

To further understand how the system performance depends on the choice of agent roles, we conducted a sensitivity analysis by varying or omitting specific roles in the A-HMAD configuration. We considered three modified settings: (i) removing one role (e.g., without the Knowledge Guru), (ii) merging the Solver and Checker roles into a single "Solver-Checker" hybrid, and (iii) duplicating one role to create partial overlap (two Checkers). Results on GSM8K and MMLU showed that removing the Knowledge Guru decreased accuracy by 2.3–2.8%, while merging Solver and Checker caused a larger drop of 3.5–4.1%. Duplicating a role yielded minimal gains (<0.5%), suggesting limited benefit from redundant specialization. These results indicate that distinct and complementary roles are important for maintaining diversity and reasoning robustness. In practice, including at least one fact-verification and one reasoning-focused agent appears crucial for optimal performance.

We see that heterogeneous debate outperforms homogeneous debate by about +5.7% on arithmetic, +6.2% on GSM8K, and +5.4% on MMLU (these numbers align with Table 1 differences). The sequential pipeline also provides gains over homogeneous (as the Checker and Guru can fix errors of the Solver), but it lags behind simultaneous debate. We hypothesize this is because in sequential, each agent only sees the immediate predecessor's output, whereas in simultaneous multi-agent debate, everyone sees everyone's ideas, allowing richer interaction. Another reason is the sequential pipeline's final answer is just the last agent's answer, whereas the debate uses consensus of all, which is more robust.

**Contribution of learned consensus** To assess our consensus module, we run A-HMAD with and without it on scenarios where agents disagree. Specifically, we took 50 GSM8K problems where in the final round not all agents had the same answer (some disagreement). We then compared what final answer would be chosen by majority vs by our learned consensus. The ground truth was known, so we compute how often each method got it correct in those cases:

- Majority vote correctness: 68% (34/50 correct by simple majority).
- Learned consensus correctness: 78% (39/50 correct when using $f_\phi$).

Thus the learned consensus improved resolution of disagreements by reducing errors by 31% relative (from 16 wrong in majority down to 11 wrong). Many of the five cases fixed were ones where a single Checker agent had a different answer (which turned out to be correct) while two Solvers agreed on a wrong answer; the learned consensus rightly favored the Checker's answer due to higher weight for that role and the Checker's confident justification that pointed out a flaw in the Solvers' reasoning. This demonstrates that $f_\phi$ successfully captures when a minority view should be trusted over a majority.

We also incorporate the learned consensus into the final accuracy numbers of A-HMAD (Table 1 already reflects it). If we hypothetically used majority vote for A-HMAD's final answers, its GSM8K accuracy would drop from 90.2 to 88.5, and MMLU from 76.5 to 74.7, according to our logs. So about 1.5–2% of the total gains are attributable to the consensus optimization.

Number of Agents and Rounds: We examine how performance scales with more agents and more debate rounds, using Arithmetic and MMLU as representative tasks (since those are quick to run in bulk).

- For agents, we tried $N = 1, 3, 5, 7$ (still using heterogeneity, so for $N > 3$ we introduced additional Solver variants or extra Checker agents to keep diversity). Accuracy on Arithmetic: 1 agent 72%, 3 agents 87%, 5 agents 90%, 7 agents 91%. There are diminishing returns beyond 5 agents. On MMLU: 1 agent 64%, 3 agents 76%, 5 agents 78%, 7 agents 78.5% (roughly saturating).
- For rounds, we ran A-HMAD with $R_{\max} = 0$ (meaning just initial answers), 1, 2, 3 on the Arithmetic dataset. Results: 0 rounds (i.e. just majority vote of initial answers) 75%, 1 round 83%, 2 rounds 87.5%, 3 rounds 89%. It appears two debate rounds capture most gains,

**Table 2** Impact of Agent Diversity (accuracy %)

| Agent Setup | Arith. | GSM | MMLU |
|---|---|---|---|
| 3 Homogeneous Solvers | 81.8 | 84.0 | 71.1 |
| 3 Heterogeneous (A-HMAD) | 87.5 | 90.2 | 76.5 |
| Sequential (Solver→Checker→Guru) | 85.0 | 87.1 | 74.0 |

and the third gives a slight boost. On MMLU, going from 1 to 2 rounds gave +4%, but 3rd round gave <1%.

These findings performance improves with more agents and rounds but with diminishing improvements per added resource. Our dynamic strategy typically chose 2 rounds, occasionally a 3rd if disagreement persisted.

Additional Analysis: We also measured the consensus rate among agents. In A-HMAD final rounds, agents reached unanimous agreement in 92% of arithmetic problems and 85% of GSM8K problems, up from 88% and 80% respectively in the homogeneous debate baseline. This suggests heterogeneity did not reduce convergence; in fact it slightly increased it, perhaps because specialized agents resolve uncertainties better. When they did not agree, our consensus module handled it as discussed.

To validate the robustness of A-HMAD, we tested it on variations of prompts and on out-of-domain questions:

- Changing the role descriptions wording did not significantly alter outcomes, indicating stability to prompt phrasing.
- On TruthfulQA (a dataset of tricky questions designed to induce hallucinations), A-HMAD (with an added agent specialized in skepticism) reduced false answers by 20% compared to single LLM. This was a small side experiment (not in main table due to space) but demonstrates generalization to other factual tasks.

### 4.6 Case study

To illustrate how A-HMAD works in practice, Fig. 1 shows an excerpt of a debate among three agents (Solver, Checker, Guru) on a sample GSM8K math problem. The Solver agent proposes an answer but makes a subtle arithmetic mistake. The Checker agent immediately catches the discrepancy ("I think you mis-added the totals"), and the Guru agent provides an alternative approach recalling a relevant formula. By the second round, the Solver corrects its answer based on the others' feedback. All three converge to the correct answer by the end of round 2. The learned consensus easily finalizes that answer. In contrast, a single agent had originally stuck with the wrong answer. This showcases the interactive error-correction that A-HMAD facilitates.

We also observed cases in factual tasks where the Knowledge Guru agent's intervention was key. For instance, on a biography question about a scientist's PhD year, the Solver and Checker both guessed a plausible year (1980) but expressed uncertainty. The Guru agent, having been prompted to recall specifics, produced "He earned his PhD in 1978." The Checker then double-checked this fact against earlier context and agreed. The final consensus picked 1978. Without the Guru role, the debate might have settled on the wrong year due to lack of specific knowledge.

In chess, our debates often had one agent enumerating moves and another evaluating them. If a Solver agent suggested a risky move, a Checker agent (with a prompt to be cautious) would raise concerns like "That move might lead to losing a rook in two moves." Then the Solver reconsidered and often found a safer move. In one puzzle, two agents debated whether a sacrifice would lead to checkmate; the debate led them to analyze three moves deep (something a single pass often fails to do), resulting in the correct move choice. This kind of deep analysis by bouncing ideas off each other is exactly what debate is intended to induce.

### 4.7 Simulated educational assistant scenario

To demonstrate the practical applicability of A-HMAD beyond synthetic benchmarks, we simulate an AI teaching assistant scenario. In this setting, multiple agents collaboratively debate how to grade or answer student queries drawn from educational QA datasets such as EdNet and ScienceQA.

Specifically, we assign distinct educational roles to three agents: a Solver agent that provides the initial answer, a Grader agent that critiques and scores the solution according to rubric guidelines, and an Explainer agent that generates constructive feedback and clarifications for the student. Through 2–3 rounds of adaptive debate, the agents refine both the correctness and pedagogical quality of responses.

Preliminary results on 50 science and math questions show that A-HMAD produces more accurate and pedagogically coherent responses than a single LLM assistant (Table 3). In human evaluation by graduate-level teaching assistants, A-HMAD outputs were rated 0.8 points higher (on a 5-point Likert scale) for factual accuracy and clarity of explanation.

This simulated teaching-assistant setting highlights A-HMAD's ability to support fact-driven educational reasoning

**Table 3** Performance comparison between A-HMAD and single-agent LLMs in the educational assistant simulation

| Model | Factual Accuracy ↑ | Pedagogical Clarity ↑ | Grading Consistency ↑ | Overall Score (5-pt Likert) ↑ |
|---|---|---|---|---|
| Single LLM (GPT-4) | 3.92 | 4.05 | 3.78 | 3.92 |
| A-HMAD (ours) | 4.72 | 4.85 | 4.60 | 4.72 |
| Δ Improvement | +0.80 | +0.80 | +0.82 | +0.80 |

Scores are averaged over 50 student queries from EdNet and ScienceQA datasets

**Table 4** Computational cost breakdown of A-HMAD across benchmarks

| Benchmark | Act. Agts | Tok./Init | Tok./Later | Rnds | Lat. (s) | Rel. Cost |
|---|---|---|---|---|---|---|
| Arithmetic | 2.1 | 210 | 180 | 2.0 | 1.1 | 1.2× |
| GSM8K | 2.5 | 340 | 270 | 2.3 | 1.4 | 1.3× |
| MMLU | 2.4 | 320 | 250 | 2.2 | 1.3 | 1.3× |
| Chess | 2.2 | 260 | 220 | 2.0 | 1.1 | 1.2× |
| Biographies | 2.7 | 390 | 310 | 2.4 | 1.5 | 1.4× |

Latency and relative cost are averaged over 100 queries per dataset

and offers a promising step toward deployable AI tutoring systems that can self-verify and collaboratively improve their answers.

### 4.8 Role contribution and interference analysis

To better understand the effectiveness of the proposed heterogeneous roles, we conducted a brief analysis of each role's contribution across tasks. We found that the Solver agent primarily drives reasoning progress, while the Checker agent plays a critical role in identifying arithmetic or logical inconsistencies. The Knowledge Guru contributes most to factual QA and biography generation tasks by injecting verifiable background information.

To mitigate potential role interference, each agent is instructed with non-overlapping responsibilities in its prompt, and the debate controller filters redundant or conflicting feedback based on discourse similarity. Furthermore, our learned consensus module adaptively adjusts each role's weight ($w_k$) during training: roles that consistently produce correct or helpful reasoning receive higher weights, whereas low-performing roles are automatically downweighted.

Quantitatively, ablation on GSM8K shows that removing the Checker reduces accuracy by 4.3%, while removing the Knowledge Guru has minimal effect on math but decreases factual QA performance by 6.1%. In qualitative analysis, we observed that the Checker was particularly valuable when catching numerical slips in Solver outputs, whereas excessive intervention from the Knowledge Guru sometimes introduced irrelevant facts in purely arithmetic settings. These findings indicate that the proposed role design contributes complementary strengths while the adaptive consensus mechanism effectively suppresses role interference.

### 4.9 Robustness and domain-specific evaluation

To further assess the robustness and practical applicability of A-HMAD beyond clean benchmark datasets, we conducted supplementary experiments on two additional settings. (1) Noisy factual QA (TruthfulQA [25]) – a dataset containing adversarially phrased or misleading factual questions. (2) Biomedical question answering (BioASQ subset) – a specialized domain requiring precise factual grounding.

Across both scenarios, A-HMAD maintained strong performance and significantly reduced factual errors compared to single-agent and homogeneous debate baselines. On TruthfulQA, A-HMAD reduced hallucinated answers by 18% relative to the standard multi-agent debate. On BioASQ, it achieved a 5% higher factual accuracy, demonstrating that the adaptive heterogeneous debate can generalize to noisy and domain-specific contexts.

These findings indicate that A-HMAD's adaptive routing and role diversity contribute to robustness against noise and domain shift, strengthening its potential for real-world deployment.

### 4.10 Computational cost analysis

To better evaluate the efficiency of A-HMAD, we report a detailed breakdown of computational costs in Table 4. The table lists the average number of active agents, token counts per debate round (initial and subsequent), average rounds per query, inference latency, and relative cost compared with MAD across benchmarks.

A-HMAD typically engages 2–3 agents per query and requires about 2 debate rounds, leading to roughly 1.25–1.3× inference time compared with MAD. The token usage grows sublinearly with the number of agents due to adaptive agent activation. While higher rounds slightly increase latency, accuracy gains saturate beyond two rounds, indicating that A-HMAD achieves most of its improvements with moderate overhead, making it feasible for real-world deployment.

## 5 Conclusions

We presented Adaptive Heterogeneous Multi-Agent Debate (A-HMAD), a framework that significantly improves factual accuracy and reasoning in language models by combining the strengths of multiple specialized agents. By allowing agents with diverse expertise to critique each other and by learning an optimal way to combine their answers, A-HMAD addresses key shortcomings of prior single-model

and multi-agent approaches. Our extensive experiments on mathematical reasoning, knowledge question answering, factual generation, and strategic game tasks demonstrate that A-HMAD achieves state-of-the-art results, outperforming standard multi-agent debate by up to 6% and single-agent methods by much larger margins. These gains come without any task-specific fine-tuning of the large language models, relying only on smarter orchestration and a small learned consensus network.

The success of A-HMAD suggests several exciting directions for future research. First, our agent roles were manually defined; an interesting avenue is to learn optimal personas or prompt styles for agents, possibly through meta-learning or reinforcement learning, to maximize debate efficacy. Second, while our consensus module was learned offline on debate data, a more integrated approach could involve end-to-end training where agents are optimized (via self-play or feedback) to reach correct consensus more often. Third, extending the notion of debate to incorporate external tools and evidence could further improve factuality – e.g., agents could jointly decide to query a database or perform experiments to resolve a disagreement. Our dynamic expert agent injection was a step in this direction, but a more general toolkit for agents to invoke could be developed. Fourth, applying A-HMAD to areas like legal or scientific QA might be fruitful, as those domains naturally benefit from dialectical discussion and evidence weighing.

From an AI safety and interpretability perspective, multi-agent debate provides a window into the reasoning process of LLMs. In our experiments, we found that debate transcripts often helped identify why a particular answer was correct or incorrect. This could be leveraged to create explanations for end-users or to detect when even the ensemble might be going astray (e.g., if agents converge but on a hallucinated fact, often one agent expresses doubt that we can monitor). Developing automatic sanity checks or uncertainty estimates from debate dynamics is an intriguing possibility. Beyond interpretability, the A-HMAD framework also shows potential for deployment in practical high-stakes domains such as legal reasoning, medical question answering, and decision-support systems, where factual reliability and transparency are essential. However, translating multi-agent debate into real-world systems introduces several challenges. First, the computational cost of coordinating multiple interacting agents can be substantial, especially for latency-sensitive applications. Second, ensuring interpretability and trustworthiness at scale remains an open problem, as users may require clear justifications for the system's collective decisions. Third, integration with external knowledge bases or retrieval systems is crucial to ground debates in verifiable facts and prevent hallucinations. In addition, we also observe cases where heterogeneity among agents may inadvertently amplify errors–for instance, when a Checker wrongly challenges a correct Solver output, leading the ensemble to converge on an incorrect conclusion. Understanding and mitigating such failure modes will be crucial for ensuring reliable deployment. Addressing these challenges–through efficient debate scheduling, explainable agent interaction design, and knowledge-grounded reasoning–constitutes promising directions for future work.

In conclusion, A-HMAD demonstrates that collaboration through constructive debate can overcome many limitations of individual AI models. As LLMs become more capable, assembling them into a thoughtful committee may be an essential strategy to ensure their answers are not only fluent, but also accurate and well-reasoned. We hope our work inspires further exploration into multi-agent approaches for building more reliable AI systems.

**Author Contributions** Yan Zhou: Methodology, Writing–original draft, Software, Formal analysis, Resources, Project administration. Yanguang Chen: Methodology, Supervision, Resources, Funding acquisition, Writing–review & editing.

**Data Availability** The datasets used in this manuscript are publicly available datasets. Detailed information about these datasets is provided in Section Dataset of this manuscript.

## Declarations

**Competing interests** The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

**Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

## References

Brown T, Mann B, Ryder N, Subbiah M, Kaplan J.D, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A et al (2020) Language models are few-shot learners. Adv Neural Inf Process Syst 33:1877–1901

Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A et al (2020) Language models are few-shot learners. Adv Neural Inf Process Syst 33:1877–1901

Kojima T, Gu SS, Reid M, Matsuo Y, Iwasawa Y (2022) Large language models are zero-shot reasoners. Adv Neural Inf Process Syst 35:22199–22213
Lin S, Hilton J, Evans O (2021) Truthfulqa: Measuring how models mimic human falsehoods. arXiv preprint arXiv:2109.07958
Ji Z, Lee N, Frieske R, Yu T, Su D, Xu Y, Ishii E, Bang YJ, Madotto A, Fung P (2023) Survey of hallucination in natural language generation. ACM Comput Surv 55(12):1–38
Ouyang L, Wu J, Jiang X, Almeida D, Wainwright C, Mishkin P, Zhang C, Agarwal S, Slama K, Ray A et al (2022) Training language models to follow instructions with human feedback. Adv Neural Inf Process Syst 35:27730–27744
Ziegler DM, Stiennon N, Wu J, Brown TB, Radford A, Amodei D, Christiano P, Irving G (2019) Fine-tuning language models from human preferences. arXiv preprint arXiv:1909.08593
Lee N, Ping W, Xu P, Patwary M, Fung P.N, Shoeybi M, Catanzaro B (2022) Factuality enhanced language models for open-ended text generation. Adv Neural Inf Process Syst 35:34586–34599
Kadavath S, Conerly T, Askell A, Henighan T, Drain D, Perez E, Schiefer N, Hatfield-Dodds Z, DasSarma N, Tran-Johnson E et al (2022) Language models (mostly) know what they know. arXiv preprint arXiv:2207.05221
Wei J, Wang X, Schuurmans D, Bosma M, Xia F, Chi E, Le QV, Zhou D et al (2022) Chain-of-thought prompting elicits reasoning in large language models. Adv Neural Inf Process Syst 35:24824–24837
Wang X, Wei J, Schuurmans D, Le Q, Chi E, Narang S, Chowdhery A, Zhou D (2022) Self-consistency improves chain of thought reasoning in language models. arXiv preprint arXiv:2203.11171
Nye M, Andreassen A.J, Gur-Ari G, Michalewski H, Austin J, Bieber D, Dohan D, Lewkowycz A, Bosma M, Luan D et al (2021) Show your work: Scratchpads for intermediate computation with language models
Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, Cao Y (2023) React: Synergizing reasoning and acting in language models. In: International conference on learning representations (ICLR)
Schick T, Dwivedi-Yu J, Dessì R, Raileanu R, Lomeli M, Hambro E, Zettlemoyer L, Cancedda N, Scialom T (2023) Toolformer: Language models can teach themselves to use tools. Adv Neural Inf Process Syst 36:68539–68551
Ji Z, Lee N, Frieske R, Yu T, Su D, Xu Y, Ishii E, Bang YJ, Madotto A, Fung P (2023) Survey of hallucination in natural language generation. ACM Comput Surv 55(12):1–38
Stiennon N, Ouyang L, Wu J, Ziegler D, Lowe R, Voss C, Radford A, Amodei D, Christiano PF (2020) Learning to summarize with human feedback. Adv Neural Inf Process Syst 33:3008–3021
Siddiky MNA, Rahman ME, Hossen M, Rahman MR, Jaman MS (2025) Optimizing ai language models: a study of chatgpt-4 vs. chatgpt-4o. Preprints. org
Kawamura K, Noelle D, Hambuchen K, Rogers T, Turkay E (2003) A multi-agent approach to self-reflection for cognitive robotics. International Conference on Advanced Robotics. Coimbra, Portugal, pp 568–575
Kawamura K, Noelle D, Hambuchen K, Rogers T, Turkay E (2003) A multi-agent approach to self-reflection for cognitive robotics. In: International Conference on Advanced Robotics, Coimbra, Portugal, pp 568–575
Kojima T, Gu SS, Reid M, Matsuo Y, Iwasawa Y (2022) Large language models are zero-shot reasoners. Adv Neural Inf Process Syst 35:22199–22213
Lee N, Ping W, Xu P, Patwary M, Fung PN, Shoeybi M, Catanzaro B (2022) Factuality enhanced language models for open-ended text generation. Adv Neural Inf Process Syst 35:34586–34599
Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, Küttler H, Lewis M, Yih W.-t, Rocktäschel T et al (2020) Retrieval-augmented generation for knowledge-intensive nlp tasks. Adv Neural Inf Process Syst 33:9459–9474
Li G, Hammoud H, Itani H, Khizbullin D, Ghanem B (2023) Camel: Communicative agents for “mind” exploration of large language model society. Adv Neural Inf Process Syst 36:51991–52008
Khan A, Hughes J, Valentine D, Ruis L, Sachan K, Radhakrishnan A, Grefenstette E, Bowman SR, Rocktäschel T, Perez E (2024) Debating with more persuasive llms leads to more truthful answers. arXiv preprint arXiv:2402.06782
Li G, Hammoud H, Itani H, Khizbullin D, Ghanem B (2023) Camel: Communicative agents for “mind" exploration of large language model society. Adv Neural Inf Process Syst 36:51991–52008
Madaan A, Tandon N, Gupta P, Hallinan S, Gao L, Wiegreffe S, Alon U, Dziri N, Prabhumoye S, Yang Y et al (2023) Self-refine: Iterative refinement with self-feedback. Adv Neural Inf Process Syst 36:46534–46594
Parrish A, Trivedi H, Nangia N, Padmakumar V, Phang J, Saimbhi AS, Bowman SR (2022) Two-turn debate doesn’t help humans answer hard reading comprehension questions. arXiv preprint arXiv:2210.10860
Minsky M (1986) Society of mind. Simon and Schuster
Du Y, Li S, Torralba A, Tenenbaum JB, Mordatch I (2023) Improving factuality and reasoning in language models through multiagent debate. In: 41st international conference on machine learning
Ouyang L, Wu J, Jiang X, Almeida D, Wainwright C, Mishkin P, Zhang C, Agarwal S, Slama K, Ray A et al (2022) Training language models to follow instructions with human feedback. Adv Neural Inf Process Syst 35:27730–27744
Zhang H, Du W, Shan J, Zhou Q, Du Y, Tenenbaum J.B, Shu T, Gan C (2023) Building cooperative embodied agents modularly with large language models. arXiv preprint arXiv:2307.02485
Wu Q, Bansal G, Zhang J, Wu Y, Li B, Zhu E, Jiang L, Zhang X, Zhang S, Liu J et al (2024) Autogen: Enabling next-gen llm applications via multi-agent conversations. In: 1st conference on language modeling
Zelikman E, Wu Y, Mu J, Goodman N (2022) Star: Bootstrapping reasoning with reasoning. Adv Neural Inf Process Syst 35:15476–15488
Schick T, Dwivedi-Yu J, Dessì R, Raileanu R, Lomeli M, Hambro E, Zettlemoyer L, Cancedda N, Scialom T (2023) Toolformer: Language models can teach themselves to use tools. Adv Neural Inf Process Syst 36:68539–68551
Guu K, Lee K, Tung Z, Pasupat P, Chang M (2020) Realm: Retrieval-augmented language model pre. Training
Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, Küttler H, Lewis M, Yih W.-t, Rocktäschel T et al (2020) Retrieval-augmented generation for knowledge-intensive nlp tasks. Adv Neural Inf Process Syst 33:9459–9474
Stiennon N, Ouyang L, Wu J, Ziegler D, Lowe R, Voss C, Radford A, Amodei D, Christiano PF (2020) Learning to summarize with human feedback. Adv Neural Inf Process Syst 33:3008–3021
Hendrycks D, Burns C, Kadavath S, Arora A, Basart S, Tang E, Song D, Steinhardt J (2021) Measuring mathematical problem solving with the math dataset. arXiv preprint arXiv:2103.03874
Minsky M (1986) Society of mind. Simon and Schuster
Wei J, Wang X, Schuurmans D, Bosma M, Xia F, Chi E, Le QV, Zhou D et al (2022) Chain-of-thought prompting elicits reasoning in large language models. Adv Neural Inf Process Syst 35:24824–24837
Wei J, Wang X, Schuurmans D, Bosma M, Xia F, Chi E, Le QV, Zhou D et al (2022) Chain-of-thought prompting elicits reasoning in large language models. Adv Neural Inf Process Syst 35:24824–24837
Li S, Du Y, Tenenbaum JB, Torralba A, Mordatch I (2022) Composing ensembles of pre-trained models via iterative consensus. arXiv preprint arXiv:2210.11522
Sabour S, Frosst N, Hinton GE (2017) Dynamic routing between capsules. Adv Neural Inf Process Syst 30

Hendrycks D, Burns C, Basart S, Zou A, Mazeika M, Song D, Steinhardt J (2020) Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300

Hurst A, Lerer A, Goucher AP, Perelman A, Ramesh A, Clark A, Ostrow A, Welihinda A, Hayes A, Radford A et al (2024) Gpt-4o system card. arXiv preprint arXiv:2410.21276

Irving G, Christiano P, Amodei D (2018) Ai safety via debate. arXiv preprint arXiv:1805.00899

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.